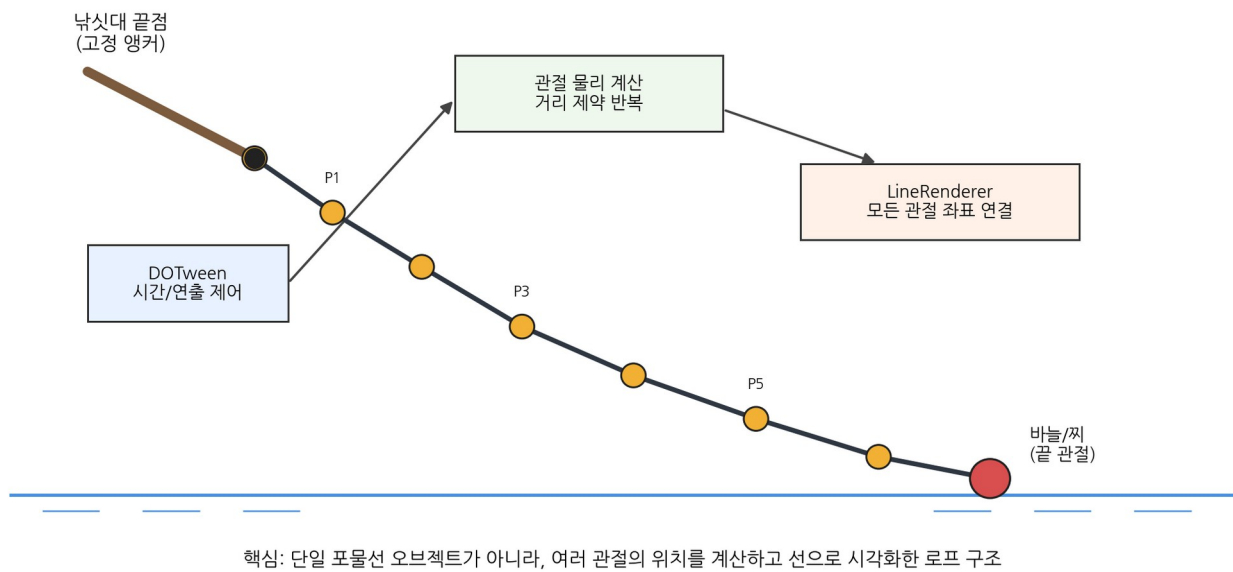


Unity 2D 낚싯줄 로프 시뮬레이션

관절 기반 물리 · LineRenderer · DOTween 통합 구현



면접 설명 및 포트폴리오 보조용 기술 문서

※ 과거 구현에 대한 기억을 바탕으로 복원한 구조이며, 정확한 클래스명과 세부 수치는 원본 코드 확인 시 조정할 수 있습니다.

1. 구현 개요

이 기능은 낚싯바늘 하나를 포물선으로 이동시키는 단순 투사체 애니메이션이 아니다. 낚싯대 끝점부터 바늘까지 여러 개의 관절 포인트를 배치하고, 각 관절의 움직임과 인접 거리 제약을 계산하여 유연한 줄의 형태를 만들었다. LineRenderer 는 계산된 관절 좌표를 순서대로 연결해 실제 낚싯줄처럼 보이게 했다.

핵심 구조

DOTween 은 “정해진 N 초”와 연출 순서를 담당하고, 관절 물리는 줄의 자연스러운 흔들림과 휘어짐을 담당하며, LineRenderer 는 그 결과를 시각화한다.

1.1 요구사항

- 낚싯줄이 직선이 아니라 여러 마디로 자연스럽게 휘어져야 한다.
- 낚싯대 끝점과 바늘 사이의 줄 길이 및 관절 간 연결성이 유지되어야 한다.
- 투척 연출은 목표 거리와 무관하게 기획에서 지정한 N 초 안에 완료되어야 한다.
- 프레임마다 계산된 관절 위치와 LineRenderer 의 정점이 정확히 동기화되어야 한다.
- 투척, 줄의 반응, 착수 효과, 낚싯대 복원 등 여러 연출을 하나의 흐름으로 제어해야 한다.

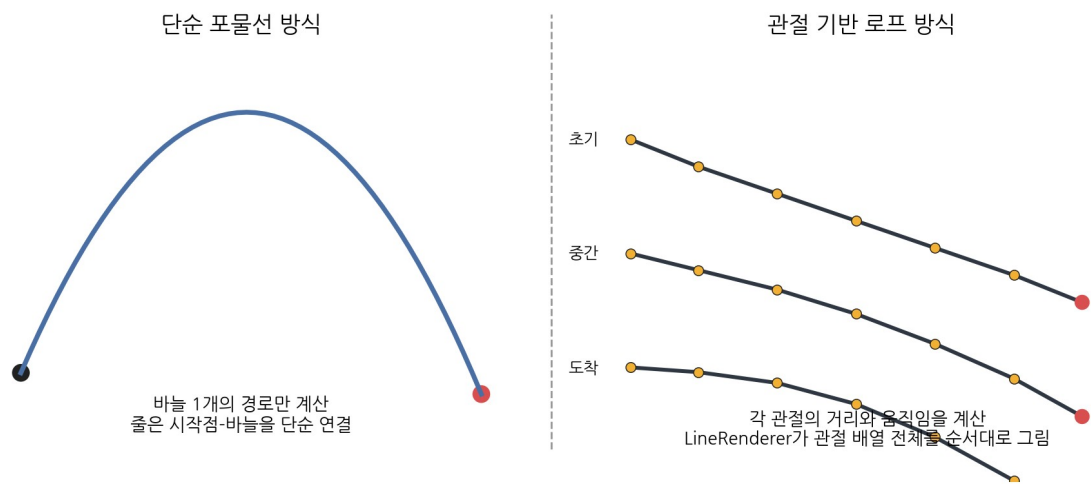


그림 1. 단순 포물선 이동과 관절 기반 로프 시뮬레이션의 차이

2. 시스템 구성

구성 요소	역할
고정 앵커	낚싯대 끝점. 첫 번째 관절의 위치를 고정하거나 매 프레임 낚싯대 끝 Transform 에 맞춘다.
관절 포인트 배열	줄을 구성하는 N 개의 포인트. 각 포인트는 현재 위치, 이전 위치 또는 속도, 고정 여부 등의 상태를 가진다.
끝 관절/바늘	마지막 관절이며 투척의 목표가 되는 포인트. DOTween 이 직접 이동시키거나 제어 목표를 갱신한다.
관절 물리 계산	중력과 관성을 적용하고, 인접 관절 사이가 일정 길이를 유지하도록 거리 제약을 반복 보정한다.
LineRenderer	positionCount 를 관절 수로 지정하고 각 관절의 월드 좌표를 SetPosition 으로 전달한다.
DOTween Sequence	투척 시간, 진행률, 낚싯대 움직임, 바늘 도착, 착수 효과 및 완료 콜백을 순차적으로 관리한다.

표 1. 주요 구성 요소와 책임 분리

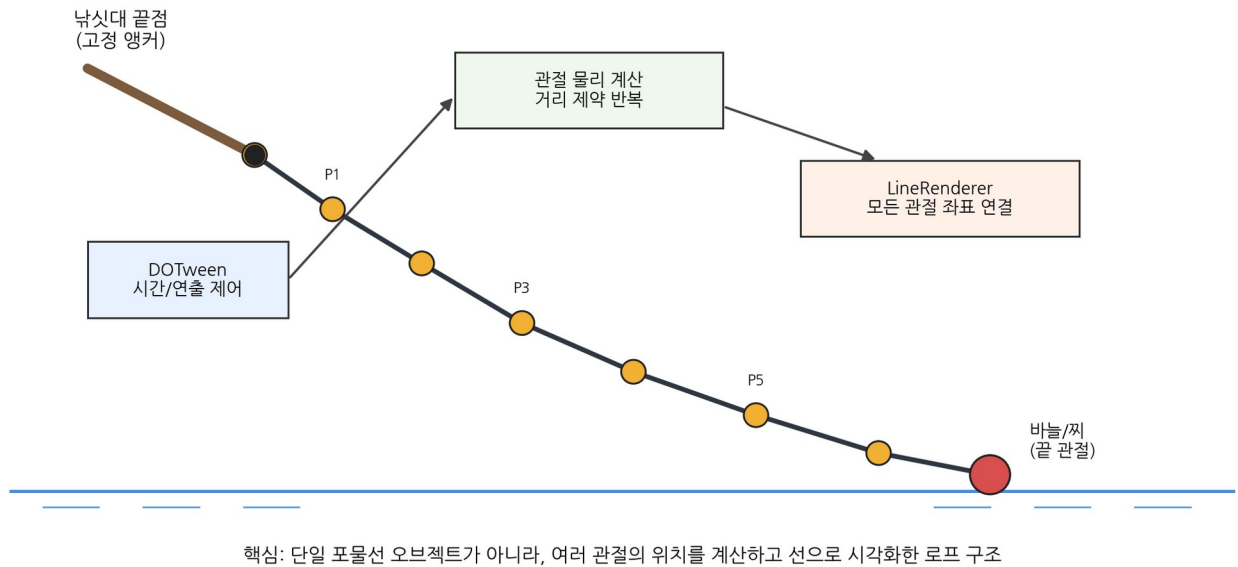
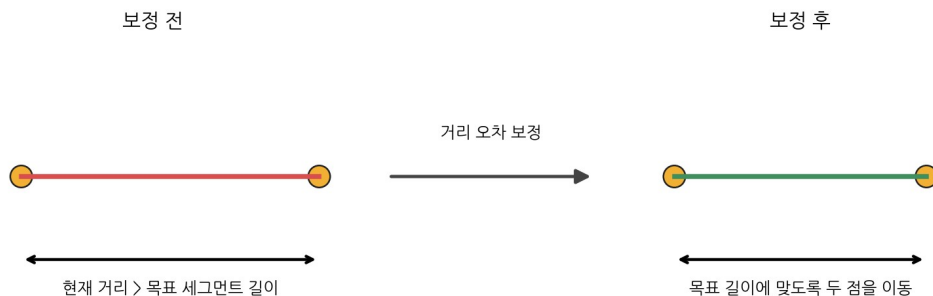


그림 2. 관절 물리, DOTween, LineRenderer의 역할 분리

3. 관절 기반 로프 물리

직접 물리 코드를 작성했다는 기억을 고려하면, 가장 유력한 방식은 각 관절을 점으로 관리하고 인접 점 사이의 거리를 제한하는 커스텀 로프 시뮬레이션이다. 구현은 Verlet 적분 또는 속도 기반 적분으로 만들 수 있으며, 핵심은 “인접 관절 사이의 목표 길이를 반복해서 만족시키는 것”이다.

인접 관절 거리 제약의 개념



이 과정을 모든 인접 관절에 여러 번 반복하면 줄 길이는 유지되면서 전체 형태는 유연하게 움직인다.

그림 3. 인접 관절의 거리 제약 보정

3.1 프레임별 처리 순서

- 고정되지 않은 각 관절에 중력과 관성을 적용한다.
- 낚시대 끝의 첫 번째 관절은 앵커 위치로 다시 고정한다.
- 모든 인접 관절 쌍의 현재 거리를 계산한다.
- 현재 거리와 목표 세그먼트 길이의 오차만큼 두 관절을 서로 당기거나 민다.
- 오차가 충분히 작아질 때까지 제약 계산을 여러 번 반복한다.
- 최종 관절 좌표를 LineRenderer에 전달한다.

복원 예시 1 - 관절 시뮬레이션

```
private void Simulate(float dt)
{
    for (int i = 1; i < points.Length; i++)
    {
        Vector2 velocity = points[i].position - points[i].previousPosition;
        points[i].previousPosition = points[i].position;
        points[i].position += velocity + gravity * (dt * dt);
    }

    for (int iteration = 0; iteration < constraintIterations; iteration++)
    {
        PinFirstPointToRodTip();
        SolveDistanceConstraints();
        ApplyControlledHookPosition();
    }
}
```

복원 예시 2 - 인접 관절 거리 제약

```
private void SolveDistanceConstraints()
{
    for (int i = 0; i < points.Length - 1; i++)
    {
        Vector2 delta = points[i + 1].position - points[i].position;
        float distance = delta.magnitude;
        float errorRatio = (distance - segmentLength) / distance;
        Vector2 correction = delta * errorRatio;

        if (!points[i].isPinned)
            points[i].position += correction * 0.5f;

        if (!points[i + 1].isPinned)
            points[i + 1].position -= correction * 0.5f;
    }
}
```

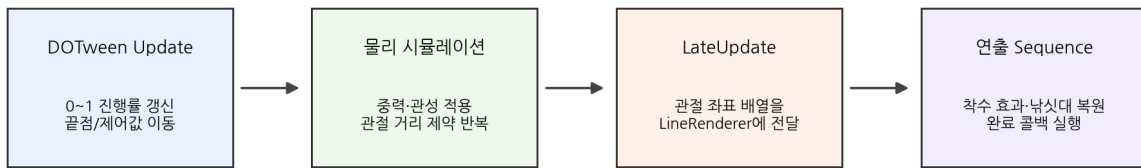
정확성 주의

실제 과거 구현이 Rigidbody2D 와 Unity Joint2D 컴포넌트를 사용했다면, 위의 커스텀 거리 제약 부분만 엔진 조인트 계산으로 대체된다. 관절 배열을 LineRenderer 로 연결하고 DOTween 으로 투척 시간을 제어하는 전체 구조는 동일하다.

4. DOTween 과 N 초 시간 제어

DOTween 의 역할은 줄 전체의 물리를 대신하는 것이 아니라, 투척 연출이 지정된 시간 안에 끝나도록 진행률과 상태 전환을 제어하는 것이다. 진행률 0 에서 1 까지를 N 초 동안 갱신하고, 이를 바늘의 제어 목표 또는 줄 길이 변화에 반영한다. 관절들은 이 제어값을 따라 물리적으로 반응한다.

프레임별 실행 흐름



정확한 N초 제어는 DOTween이 담당하고, 줄의 자연스러운 굴곡은 관절 물리가 담당한다.

따라서 “트윈 애니메이션”과 “로프 시뮬레이션”을 분리하여 각 도구가 잘하는 역할만 맡긴 구조다.

그림 4. DOTween 과 관절 물리의 프레임별 협업

복원 예시 3 - DOTween 으로 N 초 진행률 제어

```

public void Cast(Vector2 target, float duration)
{
    Vector2 start = hookControlPosition;

    castSequence?.Kill();
    castSequence = DOTween.Sequence();

    castSequence.Append(
        DOVirtual.Float(0f, 1f, duration, progress =>
        {
            // DOTween 은 시간과 제어 목표만 담당한다.
            Vector2 basePosition = Vector2.Lerp(start, target, progress);
            float arc = Mathf.Sin(progress * Mathf.PI) * castArcHeight;
            hookControlPosition = basePosition + Vector2.up * arc;
        })
        .SetEase(Ease.Linear)
    );

    castSequence.AppendCallback(OnHookReachedWater);
}
  
```

위 코드에서 포물선은 바늘 제어 목표의 이동 경로일 뿐이며, 화면에 보이는 낚싯줄의 실제 형태는 각 관절의 물리 계산 결과다. 즉 “바늘 경로”와 “줄의 모양”은 서로 다른 계층에서 처리된다.

5. LineRenderer 동기화

LineRenderer 는 물리를 계산하지 않는다. 관절 시뮬레이션이 끝난 뒤 각 관절의 최종 좌표를 순서대로 받아 선을 그리는 렌더링 계층이다. 물리 계산보다 먼저 갱신하면 한 프레임 이전 좌표가 보이거나 떨림이 발생할 수 있으므로, 일반적으로 LateUpdate 에서 처리하는 것이 안전하다.

복원 예시 4 - 관절 좌표를 LineRenderer 로 시각화

```

private void LateUpdate()
{
    lineRenderer.positionCount = points.Length;

    for (int i = 0; i < points.Length; i++)
        lineRenderer.SetPosition(i, points[i].position);
}
  
```

5.1 구현 시 발생할 수 있는 문제

문제	대응
줄이 지나치게 늘어남	세그먼트 제약 반복 횟수를 늘리거나 FixedUpdate 간격과 세그먼트 수를 조정한다.
줄이 떨리거나 폭발함	끝 관절을 강제로 이동할 때 이전 위치 또는 속도를 함께 보정하고, 한 프레임에 너무 큰 이동을 주지 않는다.
프레임마다 길이가 달라 보임	Time.deltaTime 과 FixedUpdate 사용을 구분하고 물리 계산과 렌더링 순서를 고정한다.
모바일 성능 저하	관절 수와 제약 반복 횟수를 최소화하고, 필요하지 않은 충돌 계산은 사용하지 않는다.
정확한 도착 시간이 흔들림	도착 시간은 물리 결과가 아니라 DOTween 진행률로 보장하고, 마지막 프레임에서 목표 상태를 명시적으로 고정한다.

표 2. 로프 시뮬레이션의 주요 안정화 포인트

6. 면접에서 설명하는 방법

6.1 40 초 답변

면접 답변

가장 어려웠던 구현 중 하나는 낚시줄 투척 기능이었습니다. 단순히 바늘 하나를 포물선으로 이동시킨 것이 아니라, 줄을 여러 관절 포인트로 나누고 각 관절의 물리와 인접 거리 제약을 직접 계산했습니다. 계산된 관절 좌표는 LineRenderer로 연결해 유연한 낚시줄 형태를 표현했습니다. 동시에 기획상 투척이 반드시 정해진 N 초 안에 끝나야 했기 때문에 DOTween으로 진행률과 전체 연출 Sequence를 제어했습니다. 즉 DOTween은 시간과 연출 흐름을, 커스텀 관절 물리는 줄의 자연스러운 움직임을, LineRenderer는 시각화를 담당하도록 역할을 분리했습니다.

6.2 이 사례가 어려운 구현으로 납득되는 이유

- 시간이 정확해야 하는 애니메이션 요구와 결과가 유동적인 물리 시뮬레이션을 동시에 만족시켰다.
- 여러 관절의 상태와 거리 제약을 반복적으로 계산해야 했다.
- 물리 갱신, DOTween 진행률, LineRenderer 렌더링의 실행 순서를 맞춰야 했다.
- 줄의 자연스러움, 안정성, 모바일 성능 사이에서 관절 수와 반복 횟수를 조율해야 했다.
- 하나의 도구에 모든 책임을 몰지 않고 시간 제어·물리·렌더링을 분리했다.

6.3 예상 꼬리질문

Q. 왜 Rigidbody 물리만 사용하지 않았나요?

A. 물리 결과만으로는 목표 거리와 상황이 달라질 때 정확한 N 초 완료를 보장하기 어려웠기 때문입니다. 시간은 DOTween 으로 결정하고 줄의 반응만 물리에 맡겼습니다.

Q. 왜 LineRenderer 를 사용했나요?

A. 관절 포인트 배열의 좌표를 직접 연결할 수 있어, 별도의 줄 메시를 매번 생성하는 것보다 구현과 갱신이 단순했기 때문입니다.

Q. 어떻게 줄 길이를 유지했나요?

A. 인접 관절 사이의 현재 거리와 목표 길이의 오차를 구하고 두 점을 보정하는 거리 제약을 여러 차례 반복했습니다.

Q. DOTween 은 정확히 어디에 사용했나요?

A. 0~1 진행률, 바늘의 제어 목표, 낚시대 준비·투척·착수·복원 순서 및 완료 콜백을 관리하는 데 사용했습니다.

Q. 성능은 어떻게 관리했나요?

A. 줄의 관절 수와 제약 반복 횟수를 필요한 수준으로 제한하고, LineRenderer 에는 이미 계산된 좌표만 전달했습니다.

7. 한 문장 요약

핵심 문장

여러 관절의 거리 제약을 직접 계산해 로프 형태를 만들고, LineRenderer 로 관절들을 시각적으로 연결했으며, DOTween 으로 투척이 정해진 N 초 안에 완료되도록 시간과 연출 흐름을 제어했습니다.